

Homework 06

AUTHOR
cgao19

PUBLISHED
October 4, 2026

All work is in Python. The single training/test split is drawn with seed **432** and shared by both questions. Neighbor weights are equal throughout, and plot formatting is left at library defaults.

► Show the workflow

Question 1: Compare KNN and logistic regression

The 569 rows are permuted once and the first $\text{round}(0.7 \cdot 569) = 398$ go to training, leaving 171 for testing. Each predictor is centered by its training mean and divided by its training standard deviation (divisor $n - 1$); the test rows reuse those same constants, so no test information enters the fitted models.

► Show the workflow

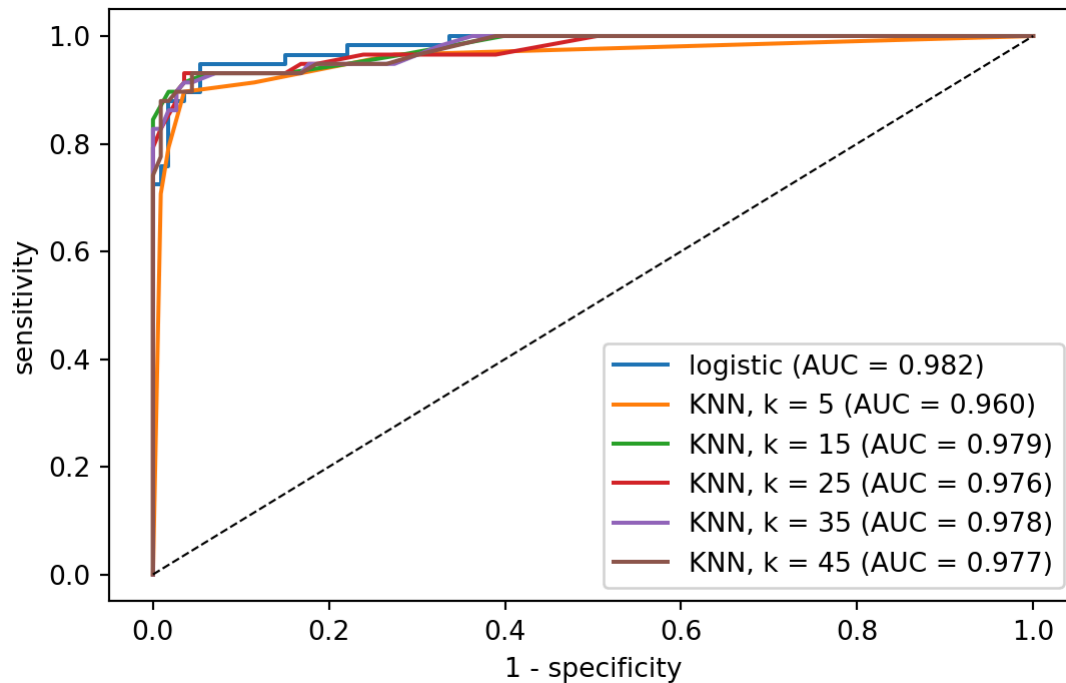
	n	malignant	malignant rate
training	398	154	0.3869
test	171	58	0.3392

Logistic regression is fit with `penalty="none"` (scikit-learn 1.0), so it is the ordinary maximum-likelihood fit with an intercept. For KNN, `predict_proba` returns the fraction of the k nearest training points that are malignant, which is the estimate of $P(Y = 1 \mid X = x)$.

► Show the workflow

	test AUC
logistic	0.9820
KNN, k = 5	0.9596
KNN, k = 15	0.9789
KNN, k = 25	0.9755
KNN, k = 35	0.9778
KNN, k = 45	0.9772

► Show the workflow



Test ROC curves for logistic regression and five KNN fits.

Preferred model: logistic regression. It has the largest test AUC, 0.982, so it ranks a malignant case above a benign one about 98% of the time. The best KNN fit is $k = 15$ at 0.979. The KNN curves improve sharply from $k = 5$ (0.960), whose estimated probabilities take only six values and are noisy, and then stay flat between 0.976 and 0.979 for k from 15 to 45. The gap between logistic regression and the best KNN is small for a test set with only 58 malignant cases, so the AUC alone does not separate them decisively. But logistic regression is at least as good, it needs no tuning of k , and with three predictors a linear log-odds boundary is evidently flexible enough here. Its coefficients also show which measurements push the probability up. KNN offers no advantage in this case to make up for being harder to interpret.

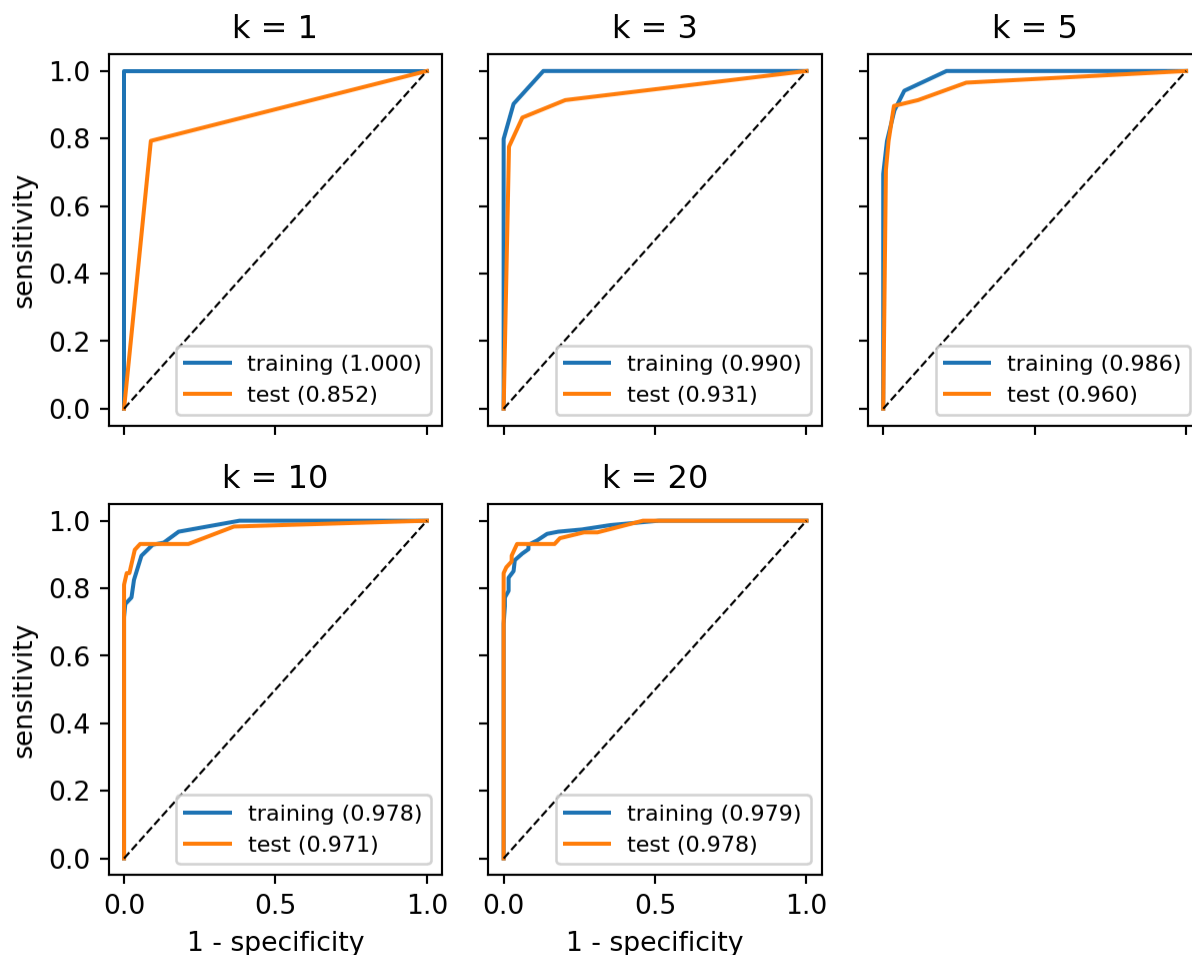
Question 2: Training and test ROC curves

The split and standardization are those of Question 1. Calling `predict_proba` on the training matrix itself makes each training point its own nearest neighbor at distance zero, which is the self-inclusion the question asks for. No two rows share all three predictor values, so for $k = 1$ every training point's only neighbor is itself.

► Show the workflow

	training	test	training - test
k			
1	1.0000	0.8523	0.1477
3	0.9903	0.9310	0.0594
5	0.9859	0.9596	0.0263
10	0.9776	0.9706	0.0069
20	0.9786	0.9777	0.0009

► Show the workflow



Training and test ROC curves for KNN, one panel per k .

What the gap shows. Training AUC is always above test AUC, and the gap shrinks quickly as k grows: 0.148 at $k = 1$, 0.059 at $k = 3$, 0.026 at $k = 5$, and nearly zero by $k = 20$. Test AUC moves the other way, rising from 0.852 to 0.978. The reason is that a training point's own label is one of the k labels averaged into its estimate. That label carries weight $1/k$, so the estimate leans toward the right answer by construction, and this advantage disappears for a new point, which is never among its own neighbors. A large k dilutes the self-vote, so training and test estimates come from essentially the same averaging and the two AUCs meet. For k between 10 and 20, training AUC no longer decreases.

At $k = 1$. Each training point's estimate is its own label, exactly 0 or 1. So every malignant case is ranked above every benign one, and the training ROC curve jumps straight to the upper-left corner with AUC exactly 1. This only says that the model has memorized the training labels; the response itself was used to make each prediction. A test point instead takes the label of one nearby training point. That estimate is also 0 or 1, so the test ROC curve has a single interior corner (sensitivity and specificity at the cutoff 0.5) joined by straight lines, and the AUC drops to 0.852. One neighbor gives the noisiest possible estimate of $p(x)$. Training performance therefore cannot be used to judge or tune k : by that measure $k = 1$ would look best, when it is in fact the worst of the five on new data.

Question 3: Peer evaluation of Homework 5

Peer-review issue: <https://github.com/r Ruiz35/stat432-fall2026/issues/2>

Part 1: Identify the item for improvement

I selected the Homework 5 report in the repository `r Ruiz35/stat432-fall12026` (`homework/week-05/homework-05.html`), **Question 4(c)**: the Python code that displays the three test images misclassified by Euclidean 5NN. The item for improvement is the grayscale display range passed to `plt.imshow`.

Part 2: Assessment

[Student to complete: exact quotation of the selected code and assessment.]

Part 3: Correction

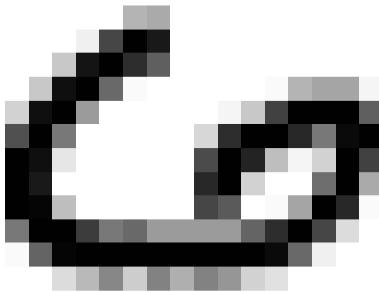
The correction keeps the report's classifier, its choice of the first three Euclidean errors, and the title format, and changes only the display range to `vmin=-1, vmax=1`. The pixel files are the Homework 5 course copies.

```
zip_train = np.loadtxt("../week-05/data/knn/zip-train.csv.gz", delimiter=",")
zip_test = np.loadtxt("../week-05/data/knn/zip-test.csv.gz", delimiter=",")
X5, y5 = zip_train[:1000, 1:], zip_train[:1000, 0].astype(int)
X5_test, y5_test = zip_test[:500, 1:], zip_test[:500, 0].astype(int)

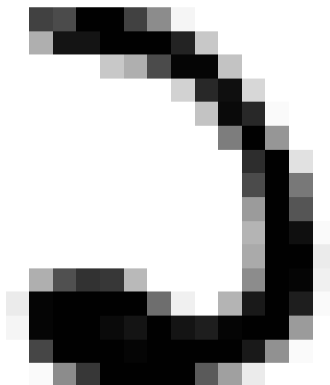
y5_hat = np.zeros(len(y5_test), dtype=int)
for i in range(len(y5_test)):
    d = np.sqrt(np.sum((X5 - X5_test[i]) ** 2, axis=1))
    neighbors = np.argsort(d, kind="stable")[:5]
    y5_hat[i] = np.argmax(np.bincount(y5[neighbors], minlength=10))

incorrect = np.flatnonzero(y5_hat != y5_test)[:3]
plt.figure(figsize=(6.5, 2.4))
for l, i in enumerate(incorrect):
    plt.subplot(1, 3, l + 1)
    plt.imshow(zip_test[i, 1:].reshape(16, 16), cmap="gray_r",
               vmin=-1, vmax=1, interpolation="nearest")
    plt.title(f"True: {y5_test[i]} Predicted: {y5_hat[i]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

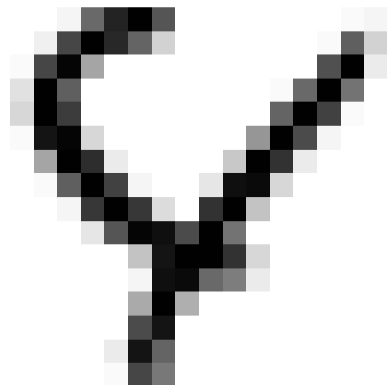
True: 6 Predicted: 0



True: 2 Predicted: 0



True: 4 Predicted: 7



Revised display of the first three Euclidean 5NN errors, with `vmin=-1` and `vmax=1`.

The three images are again test rows 4, 13, and 14 (true 6, 2, 4; predicted 0, 0, 7), so the report's written comments on these digits still apply. In the revised GitHub Issue I asked the author to set `vmin=-1, vmax=1`, regenerate the figure for the same three images, keep the true and predicted labels in the titles, and replace the figure in the report.